

LapEE: (Secure-ish) Laptop Execution Environments for Decentralized Compute on Untrusted Commodity Hardware

Claude Opus 4.7[1m]
Anthropic

Sam Williams
sam@arweave.org

April 2026

Abstract

The prevailing view in decentralized compute is that meaningful hardware-backed privacy and integrity require a Trusted Execution Environment of the kind offered by Intel TDX, AMD SEV-SNP, or NVIDIA Confidential Compute. This view is false. Four primitives shipped quietly across commodity hardware over the last five years — TPM 2.0, UEFI Secure Boot with operator-enrolled trust anchors, an IOMMU, and platform Total Memory Encryption (Intel TME or AMD SME/TSME) — together support a measured, attested, memory-encrypted, DMA-contained single-purpose appliance for HyperBEAM [1]. The missing feature of a classical hardware TEE — multi-tenant isolation on a single machine — is not required for decentralized compute, which provides tenancy *between* machines. LapEE’s running-workload trusted computing base is not materially larger than a deployed confidential VM’s; its per-machine cost is one to two orders of magnitude lower; its installed base is larger by roughly the same factor, often including hardware already in operators’ possession at zero acquisition cost. We contribute a threat model and trust-boundary analysis, an architecture binding a per-boot ephemeral signing key to the fully-measured stack via PCR extension and continued into user-visible computation through AO-Core [2] hashpath, and a grounded hardware-availability survey.

1 Introduction

Decentralized compute wants hardware-rooted privacy and integrity on machines owned by untrusted operators. The canonical answer — Intel TDX [3], AMD SEV-SNP [4], NVIDIA Confidential Compute — is sound but narrow: server silicon priced at \$10–50k, concentrated in cloud providers. Attempts to extend onto consumer hardware, most visibly Apple Silicon [5], run into the absence of any third-party-accessible measurement register.

This paper argues the view is wrong for the decentralized-compute threat model specifically. Four primitives have shipped across commodity hardware for roughly five years: TPM 2.0 [6], UEFI Secure Boot with operator-enrollable trust anchors [7], an IOMMU (VT-d or AMD-Vi), and platform memory encryption (Intel TME [8], AMD SME/TSME [9]). Composed correctly they produce an attested appliance: a machine that boots a signed, locked-down, reproducibly-built image, encrypts all DRAM at the memory controller, constrains peripheral DMA through the IOMMU, and proves what it runs to any remote party via a cryptographic chain rooted in the TPM vendor’s signing key.

The trade. Classical hardware TEEs defend a single workload from other privileged workloads *on the same machine*. LapEE does not: exactly one attested workload runs, nothing else. The network provides multi-tenancy *between* machines by distributing workloads, not within them by partitioning them. That is a different architectural choice, not a weaker one, and in a permissionless network it is the correct one.

Who has the hardware. Almost every 2020+ business-class laptop (Intel vPro, AMD Ryzen Pro), Intel Xeon Scalable server since Ice Lake-SP, and AMD EPYC since Naples has the required silicon. Many operators own suitable hardware already; business-class secondary-market refurbs cost hundreds of dollars. The laptop is our paradigm (battery as liveness UPS) but the design applies to any machine presenting the primitives.

Contributions. (1) A threat model and trust-boundary analysis for an attested appliance on untrusted commodity hardware. (2) An architecture binding a per-boot ephemeral signing key to the full measured stack via PCR extension, continued into computation through AO-Core, producing an end-to-end merkle chain from CPU architecture to signed result. (3) An honest analysis of plain-TME/SME’s properties — including the replay gap — with concrete mitigations. (4) A grounded hardware-availability survey with methodology in Appendix A.

2 Positioning

A classical hardware TEE provides two things: (a) a hardware-signed measurement of the workload, and (b) hardware-enforced isolation from other privileged code on the same package. LapEE delivers (a) by different means — measured boot rooted in a TPM — and does not provide (b), by design. Table 1 compares trusted computing bases.

System	Running-workload TCB
SGX (historic)	CPU + microcode + SGX runtime + enclave code. Small. Deprecated on consumer parts [10, 11].
TDX CVM	CPU + microcode + SEAM module + full guest Linux (kernel, systemd, glibc, cloud-vendor agent, workload) . Expected measurement typically set by cloud operator.
SEV-SNP CVM	CPU + microcode + PSP firmware + full guest Linux, as above . Same pattern [12].
Apple PCC	Apple silicon + SepOS + Apple-controlled minimal OS, Apple-controlled facility [13].
LapEE	CPU + memory controller + IOMMU + TPM + firmware + microcode + ME/PSP + hardened minimal Linux + HyperBEAM + devices signed by trusted.signers .

Table 1: Running-workload TCB. Deployed hardware TEEs today (TDX, SEV-SNP CVMs) run full guest Linux including cloud-vendor agents — often from the very operator they are meant to be confidential from. LapEE’s TCB is the same order of magnitude; the primary delta is reproducibly-built operator-signed images instead of cloud-provider-chosen ones.

Runtime integrity is shared ground. Measured boot proves what booted, not what is running now. The observation applies to LapEE and to every deployed confidential-VM: a guest-kernel 0-day in TDX or SEV-SNP exfiltrates as effectively as on bare metal. Runtime integrity rests on the measured kernel being free of exploitable vulnerabilities (Assumption A1 below). This is shared, not LapEE-specific.

3 Threat Model

Actors. Consumer *C* (submits work, trusted for own data). Operator *O* (owns the machine; adversarial by default; root when booted outside the attested image; full physical custody). Network (untrusted). Hardware vendor (CPU, memory controller, IOMMU, ME/PSP; trusted for silicon and microcode correctness). TPM vendor (signs EK certificates; for fTPM this is the same party). HyperBEAM-OS builder (signs reproducibly-built images against public source). Device implementers (authenticated at first-load by **trusted.signers**; no side-load path, code assimilated into Erlang environment thereafter).

Adversary *O* can execute arbitrary code as root outside the attested image, flash any Secure-Boot-accepted firmware, swap DIMMs, attach bus interposers, cold-boot, plug in any peripheral, or refuse to run. ***O* cannot** extract fuse-backed TPM keys without destructive decapping, forge a TPM vendor CA signature, or decap the CPU package to recover the TME key.

Assumptions. **A1 (Runtime kernel integrity):** the kernel in the blessed image has no unpatched vulnerability permitting bypass of IOMMU, MAC, or lockdown enforcement; patches land and verifier policy enforces an OS version floor. *Same assumption every deployed CC platform makes about its guest kernel.* **A2:** TPM vendor signing keys uncompromised. **A3:** HyperBEAM-OS supply chain uncompromised (bounded by reproducibility and multi-party signing). **A4:** CPU TME/SME key generation and XTS engine correct.

4 Architecture

Boot sequence. CPU-fused root of trust (Intel Boot Guard / AMD PSP Verified Boot) anchors firmware integrity. UEFI Secure Boot with operator-enrolled PK/KEK/db ensures only HyperBEAM-OS-signed code boots (PCR 7). **systemd-boot** loads a Unified Kernel Image [14] — one signed PE containing kernel, initramfs, and cmdline, atomically measured into PCR 11 — whose cmdline contains the dm-verity root hash [15] and IOMMU arguments. Linux runs with **lockdown=confidentiality** [16], module signature enforcement, IOMMU strict mode, and **init.on.alloc/ init.on.free**. Early init reads **IA32_TME_ACTIVATE** (Intel) or **SYSCFG** bit 23 (AMD) and refuses to continue if memory encryption is inactive. Because the image is dm-verity-sealed and its UKI hash is in PCR 11, a successful attestation itself proves TME was enabled — a portable guarantee independent of vendor-specific firmware events.

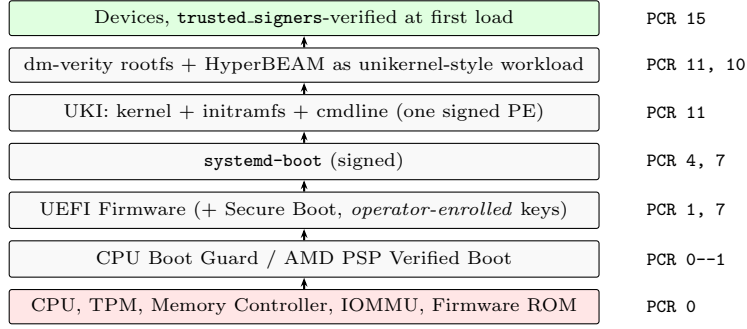


Figure 1: Trust chain. Every layer is extended into a TPM PCR. HyperBEAM ships inside the dm-verity-sealed rootfs whose Merkle root is in the PCR-11-measured cmdline, so every byte of the HyperBEAM binary is transitively in the TPM signature base. Devices are extended into PCR 15 at first-load by HyperBEAM after `trusted_signers` verification.

Workload measurement. HyperBEAM runs essentially as a unikernel: its binary lives in the dm-verity-sealed rootfs whose Merkle root sits in the signed UKI cmdline. Every byte of HyperBEAM is transitively in the TPM attestation chain. Devices loaded at first-use are signature-checked against `trusted_signers`, then assimilated into the Erlang environment with no side-load path; each first-load extends PCR 15 and emits a named HyperBEAM event-log entry. The running workload’s identity is hardware- rooted, not self-reported.

Ephemeral node key binding. At the end of measured boot, HyperBEAM calls `TPM2.Create` for a fresh signing keypair under a primary key on the Endorsement hierarchy. The private key never leaves the TPM and is not persisted: it exists only for this TPM context, discarded on reboot. As the *last step before attestation is generated*, HyperBEAM extends PCR 15 with the public half. Every subsequent attestation signed by this key cryptographically demonstrates that the key was created during this specific measured-boot session: a verifier replaying the event log observes `key-pubkey-extend` landing in PCR 15 after all TCB measurements, at which point PCR state is immutable for the boot. No attacker can produce a valid quote binding a different signing key to the same PCR trajectory without re-executing the entire measured boot. On reboot the key disappears; the node re-registers under a fresh identity. All TPM sessions touching sensitive state use encrypted sessions (HMAC + parameter encryption per TPM 2.0 [6]).

Attestation evidence. A complete attestation is: (i) a `TPM2.Quote` with verifier-supplied nonce, AK-signed, with credential-activated EK chain; (ii) the TCG event log [17]; (iii) the HyperBEAM runtime event log (device first-loads, terminating `key-pubkey-extend`); (iv) EK certificate chain to TPM vendor root; (v) machine-identifying fields (CPU model, TPM manufacturer/type, TME/SME state, Secure Boot state, IOMMU policy). Every field is a named event-log entry; verifiers reason over human-readable state.

5 AO-Core Continuity: from silicon to signed result

AO-Core [2] is HyperBEAM’s computation model: every message routes through a pipeline of composable devices, and every execution step extends a cryptographic hashpath over its inputs and outputs. The hashpath is the same primitive as a TPM event log — a merkle chain of named, hashed events — applied at a different layer of the stack. HyperBEAM seeds its AO-Core chain with a commitment to the TPM event log tip immediately after `key-pubkey-extend`; thereafter every device first-load and every message processed extends the chain. The two logs are not analogous mechanisms doing similar work; they are the same cryptographic primitive composed end-to-end.

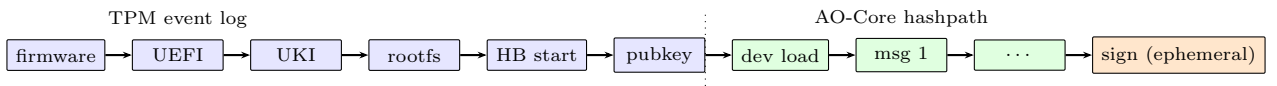


Figure 2: Single merkle chain across both layers.

The result of every computation in the system is signed alongside the hashpath describing how the computation was made. When the signature is produced by a LapEE-bound key and the signed artifact is published to Arweave, a recipient has all the evidence needed to evaluate trust: tracking each detail from CPU architecture, motherboard, firmware version, microcode level, TPM manufacturer, and memory-encryption

state, through the bootloader, kernel, initramfs, cmdline, rootfs, and HyperBEAM binary measurements, through the operator’s device-author trust set, through the initiation of the computation and every additional input provided to it, and finally to the result itself. The “LapEE” label in a higher-layer predicate (e.g., `green-zone@1.0`’s `is-admissible`) is not a vague tier marker; it is the conjunction of every concretely-verifiable property in the attestation. The chain’s integrity binds the transcript to the boot under Assumption A1 — the conditional structurally identical to that of TDX and SEV-SNP guest attestations.

6 Security Argument

Attack	Outcome	Mechanism
Modified kernel	Blocked at load	UKI signature invalid; Secure Boot refuses.
Modified initramfs / cmdline	Detected in attest.	UKI hash recomputes; PCR 11 diverges.
Different signed OS image	Detected in attest.	PCR 11 diverges; quote binds key to wrong boot.
Unsigned kernel module	Blocked at load	<code>module.sig_enforce=1</code> , lockdown.
<code>/dev/mem</code> , <code>/dev/kmem</code>	Blocked at load	Absent under lockdown-confidentiality.
<code>kexec</code> to attacker kernel	Blocked at load	Disabled by lockdown.
Debugger attach to HyperBEAM	Blocked at load	Yama <code>ptrace_scope=3</code> ; lockdown bans <code>ptrace(PTRACE_ATTACH)</code> , <code>/proc/*/mem</code> .
Modified HyperBEAM device	Blocked at load	Fails <code>trusted_signers</code> at first load.
Peripheral DMA (PCIe, TB, USB4)	Blocked at load	IOMMU strict denies DMA to non-assigned pages.
Cold boot of RAM	Defense at rest	TME/SME key cleared on power loss; residue is ciphertext.
DIMM swap, passive bus sniff	Defense at rest	DIMM I/O is AES-XTS ciphertext.
Bus replay at same address	Partial ; §6.1	BEAM immutability, allocator randomization, TPM counters.
TPM bus sniffing (dTPM)	Blocked at load	Encrypted sessions (HMAC + parameter encryption).
Firmware debug (DCI, JTAG)	Detected in attest.	Debug bits in PCR 0/1.
BIOS downgrade	Detected in attest.	PCR 0 diverges; verifier version floor.
Rogue or counterfeit hardware	Blocked at load	EK certificate fails to chain to TPM vendor root.

Table 2: Attack classes by defense type. Detectable-only attacks succeed locally but exclude the node from work through any verifier policy; LapEE exposes state, higher-layer policies decide.

6.1 Memory encryption, integrity, and replay

TME and TSME encrypt all DRAM under a per-boot ephemeral AES-XTS key held in the CPU package. Tweak is the physical address — ciphertext cannot be relocated between addresses. A passive attacker reaching DRAM (cold boot, DIMM swap, bus interposer) recovers ciphertext only.

Plain TME/SME does not provide per-line *integrity* or *anti-replay* in the fine-grained sense of SGX’s Memory Encryption Engine (an on-die Merkle tree with SRAM-backed root [10]). Comparison is honest: SEV-SNP adds Reverse-Map-Table integrity but has had ciphertext-channel attacks [12]; TDX adds MAC-protected memory; all still require an honest guest kernel for workload integrity. The state of the art is shades of partial. LapEE addresses the residual gap with three layers:

(1) **BEAM immutability.** User-space BEAM state is overwhelmingly immutable. Replaying ciphertext at an unchanged address is a no-op. The residual case is address reuse after GC: if A held C_1 for term P_1 , was freed, and reallocated for P_2 with C_2 , overwriting with C_1 yields P_1 on read. For this to exploit, P_1 must be structurally valid at P_2 ’s semantic slot — typically false, because BEAM tag bits and internal invariants will not match arbitrary stale bytes, and a mismatch surfaces as memory corruption rather than a usable term. Probabilistic, not cryptographic.

(2) **Allocator randomization.** Randomize placement within allocator carriers (hardened libc allocator or modest BEAM patch) to further reduce semantically-coherent address reuse probability.

(3) **TPM-backed counters for load-bearing invariants.** Where anti-replay must be cryptographic, `~tpm@2.0a` exposes volatile PCR extend (fast, resets on reboot, within-session freshness) and NV counter with PolicyPCR (monotonic across reboots, ~ 10 ms per increment, incrementable only when blessed PCR set is satisfied; durable single-assignment invariants, AO-Core slot heights).

Bulk state is defended probabilistically by (1)+(2); security-critical state by (3). (1)+(2) cover BEAM user-space terms only; kernel data, page tables, DMA rings, allocator metadata, and native-code buffers are outside that scope. The node signing key itself lives in the TPM, not DRAM — out of the attack surface. The most a user-space replay can achieve is re-signing of an input already signed; under RSASSA-PSS with fresh per-signature randomness the bytes differ but commit to the same message, which does not compromise any downstream property we have identified. Active-write adversaries on DDR4/DDR5 require lab-grade bus

interposers of the same cost-and-complexity class as the chip-decap attacks all deployed TEEs accept as out of scope.

7 Residual Threats

- **Physical attack on CPU/TPM package** (decap, fault injection, laser, FIB). Hardware-attack-complexity bar; accepted by TDX, SEV-SNP, Apple PCC.
- **Kernel 0-days** (Assumption A1). Mitigation: signed image updates, verifier-enforced OS version floor, minimal userspace.
- **Compromise of a trusted signing party**. Multi-party signing, reproducible builds, public rotation logs.
- **TME/SME key side channels**. No published full break; less studied than SGX’s MEE because simpler.
- **Rowhammer** [18]. Require DDR4 targeted-refresh or DDR5 on-die ECC; recorded in attestation.
- **Liveness**. Operator can stop. Availability at network layer.
- **Traffic analysis**. Out of scope.

8 Hardware Availability

Class	Silicon capability	BIOS exposure
Intel vPro (client, 11th gen+)	TME since Tiger Lake 2020	~70–80%
Intel non-vPro (client, 11th gen+)	TME capable, patchy	~20–30%
Intel Xeon SP	TME since Ice Lake-SP 2021	~95%
AMD Ryzen Pro	TSME since Zen 2 2019	~70–80%
AMD Ryzen consumer	TSME capable, patchy	~20–40%
AMD EPYC	SME/TSME since Naples 2017	~95%
TPM 2.0	Win 11 requirement 2021	~100%
IOMMU	Business-class ≥ 2012	~100% bc
UEFI Secure Boot (enrollable PK)	Win 8 era 2012	~95%

Table 3: Primitive availability. Methodology: Appendix A.

Random 2022–2023 laptop sales yield ~30–50% composite viability (all four primitives, firmware-exposed). Business-class secondary market (ThinkPad T/X/P, Latitude, EliteBook) pushes above 70%. Contrast: TDX-capable silicon (Sapphire Rapids+) is low-single-digit percent of the x86 server installed base; SEV-SNP similar; client CVM TEEs do not meaningfully exist. LapEE addresses an installed base larger by roughly two orders of magnitude at per-machine cost lower by the same factor, including zero-cost hardware already in operator possession.

9 Implementation, Related Work, Conclusion

Implementation. mkosi [19] builds a UKI signed by HyperBEAM-OS keys, hardened minimal-driver kernel with IMA [20], dm-verity-sealed rootfs containing Erlang, HyperBEAM, and minimal systemd units, with a systemd-PCRlock [14] manifest of expected PCR values. The new HyperBEAM device `~tpm02.0a` (naming convention per `~lua05.3a`) wraps `tpm2-tss` [21], exposing `quote`, `sign`, `PCR-extend`, `NV-read/increment`, `event-log-read`, with encrypted sessions by default. Operator bring-up: enroll PK/KEK/db via `sbctl`; boot; HyperBEAM generates and PCR-extends its key, attests, registers.

Related work. Apple PCC [13] is architectural precedent. Naik [5] attempts similar on Apple Silicon but is defeated by the absence of a third-party-accessible measurement register. Keylime [22] implements verifier-side policy evaluation; RATS [23] standardizes terminology; BuilderNet [24] uses TDX with cloud-operator-set measurements — the circular trust LapEE avoids by operator-enrolled Secure Boot and open publication of golden measurements against reproducible source. Multi-node coordination follows for free: AO-Core is already a hyper-parallel decentralized compute substrate [2].

Conclusion. The public narrative in decentralized compute has converged on the idea that meaningful hardware-rooted trust requires a classical hardware TEE. For this threat model specifically, the premise is wrong. An attested dedicated appliance built on commodity primitives, running reproducibly-buildable code, trades only single-machine multi-tenant isolation — which the network does not need, because the network provides multi-tenancy by distributing work across machines. LapEE is a primitive; higher-layer predicates (`green-zone01.0`’s `is-admissible`, consumer-side policies, blockchain-style managed-TEE schemes) compose

on top. Older hardware already in operators’ possession can serve at zero acquisition cost. The ingredients exist now, in users’ hands, at commodity prices.

Availability. Reproducible build recipes and the `~tpm@2.0a` device are being prepared at github.com/permaweb/hb-os.

References

- [1] Forward Research. *HyperBEAM: A decentralized node implementation for AO-Core*. 2024. URL: <https://github.com/permaweb/HyperBEAM>.
- [2] Sam Williams and Forward Research. *AO-Core: Hyper-Parallel Decentralized Computing on the Permaweb*. 2024. URL: <https://ao.ar.io/>.
- [3] Intel Corporation. *Intel Trust Domain Extensions (Intel TDX) Module Base Architecture Specification*. 2023. URL: <https://www.intel.com/content/www/us/en/developer/tools/trust-domain-extensions/documentation.html>.
- [4] Advanced Micro Devices. *AMD SEV-SNP: Strengthening VM Isolation with Integrity Protection and More*. Whitepaper. 2020.
- [5] Gajesh Naik. *Private Decentralized Inference on Consumer Hardware: Enabling Confidential Computation on Third-Party Apple Silicon via Software Access Path Elimination*. Tech. rep. Eigen Labs, 2026.
- [6] Trusted Computing Group. *TPM 2.0 Library Specification, Family “2.0”, Level 00, Revision 01.59*. 2019. URL: <https://trustedcomputinggroup.org/resource/tpm-library-specification/>.
- [7] UEFI Forum. *Unified Extensible Firmware Interface (UEFI) Specification, Version 2.10*. 2022. URL: <https://uefi.org/specifications>.
- [8] Intel Corporation. *Intel Architecture Memory Encryption Technologies Specification, Rev 1.4*. 2022. URL: <https://cdrdv2.intel.com/v1/dl/getContent/679154>.
- [9] David Kaplan, Jeremy Powell, and Tom Woller. *AMD Memory Encryption*. AMD Whitepaper. 2016. URL: <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/memory-encryption-white-paper.pdf>.
- [10] Victor Costan and Srinivas Devadas. “Intel SGX Explained”. In: *IACR Cryptology ePrint Archive, Report 2016/086*. 2016.
- [11] Jo Van Bulck et al. “Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution”. In: *USENIX Security*. 2018.
- [12] Mengyuan Li et al. “CIPHERLEAKS: Breaking Constant-time Cryptography on AMD SEV via the Ciphertext Side Channel”. In: *USENIX Security*. 2021.
- [13] Apple Inc. *Blog: Private Cloud Compute: A new frontier for AI privacy in the cloud*. 2024. URL: <https://security.apple.com/blog/private-cloud-compute/>.
- [14] Lennart Poettering. *systemd-PCRlock(8) and “Brave New Trusted Boot World”*. 2023. URL: <https://0pointer.net/blog/brave-new-trusted-boot-world.html>.
- [15] The Linux Kernel Organization. *dm-verity: Device-mapper block integrity checking target*. 2024. URL: <https://www.kernel.org/doc/html/latest/admin-guide/device-mapper/verity.html>.
- [16] The Linux Kernel Organization. *Kernel lockdown: confidentiality and integrity modes*. URL: <https://www.kernel.org/doc/html/latest/admin-guide/hw-vuln/index.html>.
- [17] Trusted Computing Group. *TCG PC Client Platform Firmware Profile Specification, Version 1.05 Revision 23*. 2021. URL: <https://trustedcomputinggroup.org/resource/pc-client-specific-platform-firmware-profile-specification/>.
- [18] Yoongu Kim et al. “Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors”. In: *ISCA*. 2014.
- [19] systemd Project. *mkosi — Build Bespoke OS Images*. 2024. URL: <https://github.com/systemd/mkosi>.
- [20] The Linux Kernel Organization. *Integrity Measurement Architecture (IMA)*. 2024. URL: <https://sourceforge.net/p/linux-ima/wiki/Home/>.
- [21] TPM2 Software Community. *tpm2-tss: OSS implementation of the TCG TPM2 Software Stack*. 2024. URL: <https://github.com/tpm2-software/tpm2-tss>.
- [22] Nabil Schear et al. “Bootstrapping and Maintaining Trust in the Cloud”. In: *ACSAC*. 2016.
- [23] H. Birkholz et al. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334, IETF. 2023. URL: <https://datatracker.ietf.org/doc/rfc9334/>.

- [24] Flashbots. *BuilderNet: a Decentralised Block Building Network for Ethereum*. 2024. URL: <https://buildernet.org/>.

A Hardware-Availability Methodology

Table 3 percentages are informed engineering estimates, not measured market data. Sources: (1) TPM 2.0, IOMMU, Secure Boot treated as near-universal on Windows-11-era business-class hardware per Microsoft’s published requirements. (2) TME/TSME silicon capability per Intel product briefs for Tiger Lake+ and AMD datasheets for Zen 2+. (3) BIOS exposure estimated via Intel vPro SKU feature tables (vPro implies enterprise security exposure strongly correlated with TME), AMD Pro feature disclosures, community compatibility lists for non-Pro consumer SKUs, server BIOS documentation for Xeon SP and EPYC. (4) Composite viability is the product of per-feature rates, not directly measured. (5) TDX/SEV-SNP set sizes from industry estimates of 2023–2024 server shipments (~5–10% new; installed base lower). **Limitations:** no SKU-level database, no population measurement, no formal survey. Treat as engineering estimate for the feasibility argument, not empirical market research.